Dynamic Programming — Complete Revision Notes

Date: 19Jul26 | Full LaTeX Transitions, Visual Intuition & Original C++ Implementations

Contents

1 Dice Combinations 2

2 Min Coins 2

3 Coin Combinations 1 3

4 Coin Combinations 2 3

5 Removing Digits 4

6 Grid Paths 4

7 Bookshop 5

8 Array Description 6

9 Counting Towers 7

10 Edit Distance 7

11 LCS (Longest Common Subsequence) 8

12 Rectangle Cutting 9

13 Minimal Grid Path 10

14 Money Sums 11

15 Removal Game 12

16 Two Sets 2 14

17 Mountain Range 14

18 Increasing Subsequence 15

19 Projects 16

20 Elevator Rides 17

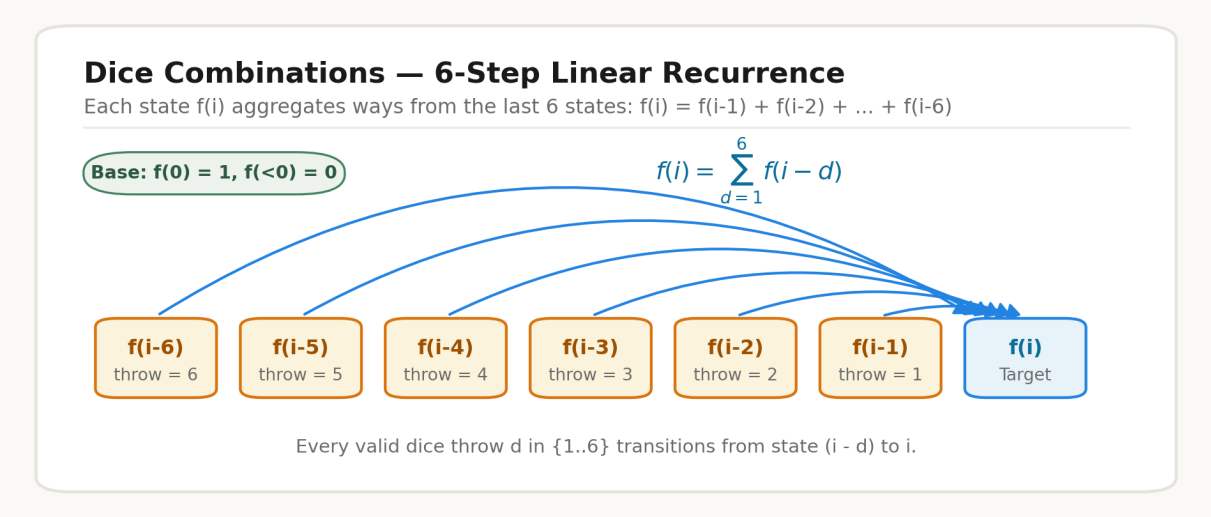
21 Counting Tilings 18

22 Counting Numbers 21

23 Increasing Subsequences 2 23

1 Dice Combinations

Goal: Count ordered ways to make sum n using dice throws $\{1, 2, 3, 4, 5, 6\}$ repeatedly.

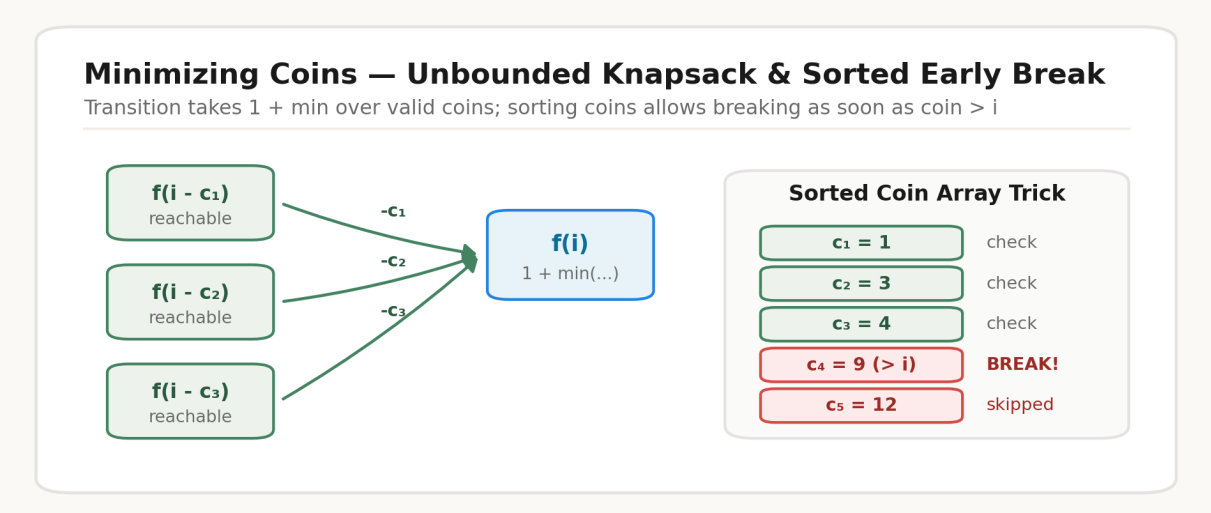


- **Definition**: $f(i)$ = number of ordered ways to make sum i.
- **Base Cases**: $f(0) = 1$, and $f(i) = 0$ for all $i < 0$.
- **Transition**:

$$f(i) = \left(\sum_{\text{throw}=1}^{6} f(i - \text{throw}) \right) \bmod (10^9 + 7)$$

2 Min Coins

Goal: Minimum number of coins from $a(i)$ (with unlimited repetition) needed to make sum n.



- **Definition**: $f(i)$ = minimum number of coins needed to make sum i.
- **Base Cases**: $f(0) = 0$, and $f(i) = +\infty$ for $i < 0$ (and unreachable sums).
- **Transition**:

$$f(i) = 1 + \min\{\, f(i - \text{coin}) \mid \text{coin} \in \mathcal{C},\ i - \text{coin} \geq 0 \,\}$$

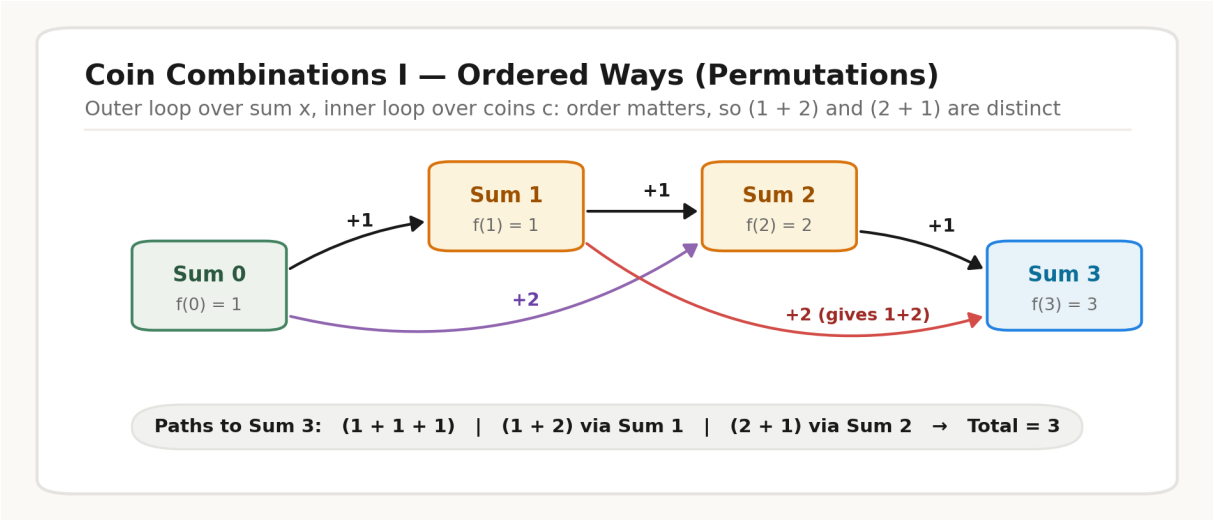
- **Implementation Trick**: **Sort** the coins in ascending order so you can `break` early as soon as the current coin exceeds i (since all subsequent coins are also useless).

3 Coin Combinations 1

Goal: Count distinct *ordered* ways (permutations) to make sum n from $a(i)$.

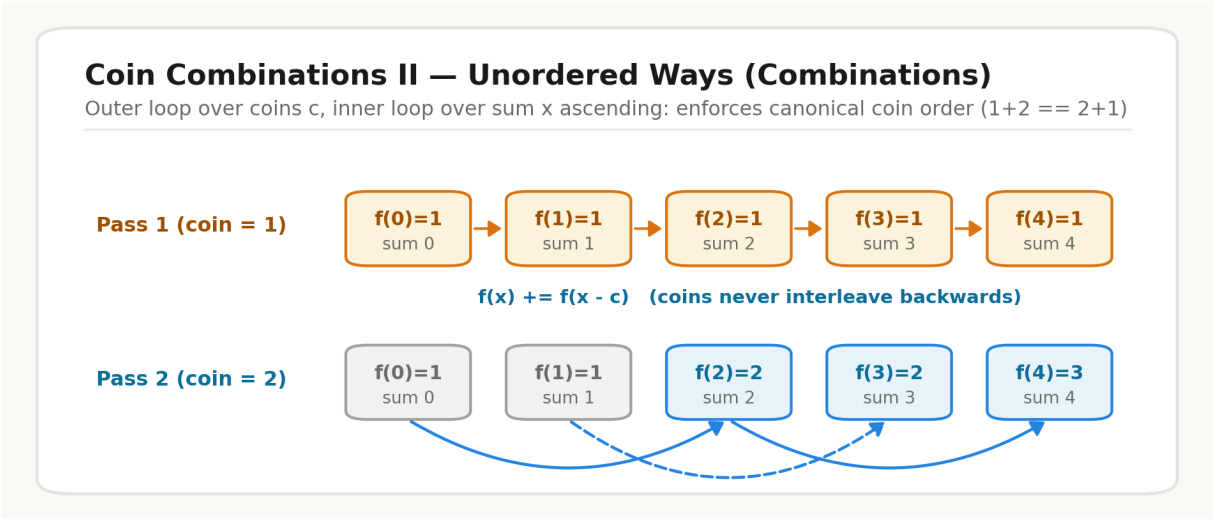
- Same structure as *Dice Combinations*: $1+2$ is counted as distinct from $2+1$.
- **Definition**: $f(i)$ = number of ordered ways to make sum i.
- **Base Cases**: $f(0) = 1$, $f(i) = 0$ for $i < 0$.
- **Transition** (*outer loop over sum i, inner loop over coin c*):

$$f(i) = \left(\sum_{c \in \mathcal{C},\ i-c \geq 0} f(i-c) \right) \bmod (10^9 + 7)$$



4 Coin Combinations 2

Goal: Count distinct *unordered* combinations to make sum n using $a(i)$ ($1+2$ is the same as $2+1$).



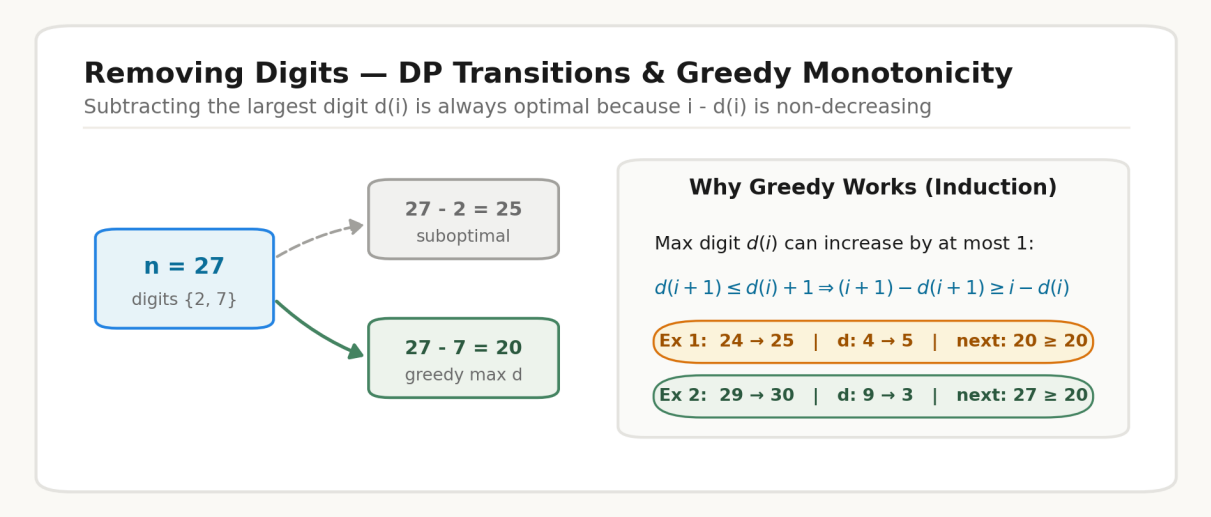
- **Core Idea**: Sort and loop over **coins first** (outer loop), then loop over sum x ascending (inner loop).
- **Definition**: $f(x)$ = number of ways to form sum x.

- **Base Case**: $f(0) = 1$.
- **Transition** (*for each coin c, for sum $x = c \dots n$ ascending*):

$$f(x) \mathrel{+}= f(x-c) \pmod{10^9+7} \quad \text{subject to } x - c \geq 0$$

5 Removing Digits

Goal: Given n, at each step transition $n' = n - d$ where d is a non-zero digit of n. Find minimum steps to reach 0.



- **Definition**: $f(i) =$ minimum steps to reduce i to 0.
- **Base Case**: $f(0) = 0$.
- **Transition** (*computed iteratively for $i = 1 \dots n$*):

$$f(i) = 1 + \min\{\, f(i-d) \mid d \text{ is a non-zero digit in } i \,\}$$

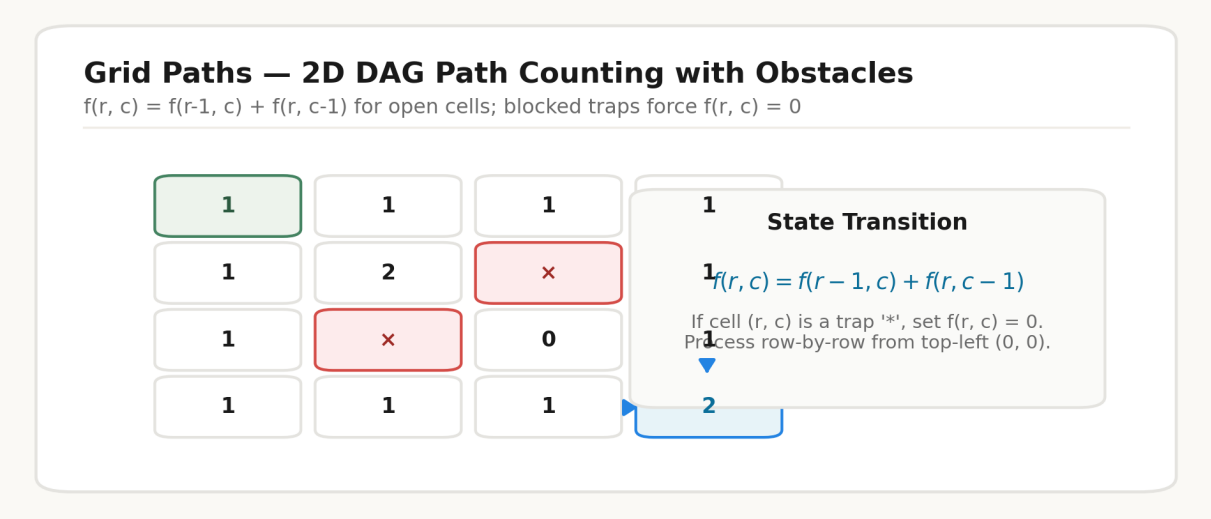
- **Alternative Greedy Solution**: Always subtract the largest digit $d(i) = \max(\text{digits}(i))$ because $f(i)$ is monotonically non-decreasing:

$$d(i+1) \leq d(i) + 1 \implies (i+1) - d(i+1) \geq i - d(i)$$

  Since the greedy next state is non-decreasing (e.g. $24 \to 25$ changes $d : 4 \to 5$, next state $20 \geq 20$; $29 \to 30$ changes $d : 9 \to 3$, next state $27 \geq 20$), induction confirms $f(i)$ is non-decreasing and subtracting $d(i)$ is globally optimal.

6 Grid Paths

Goal: Count paths from top-left to bottom-right in a 2D grid with blocked cells (*).

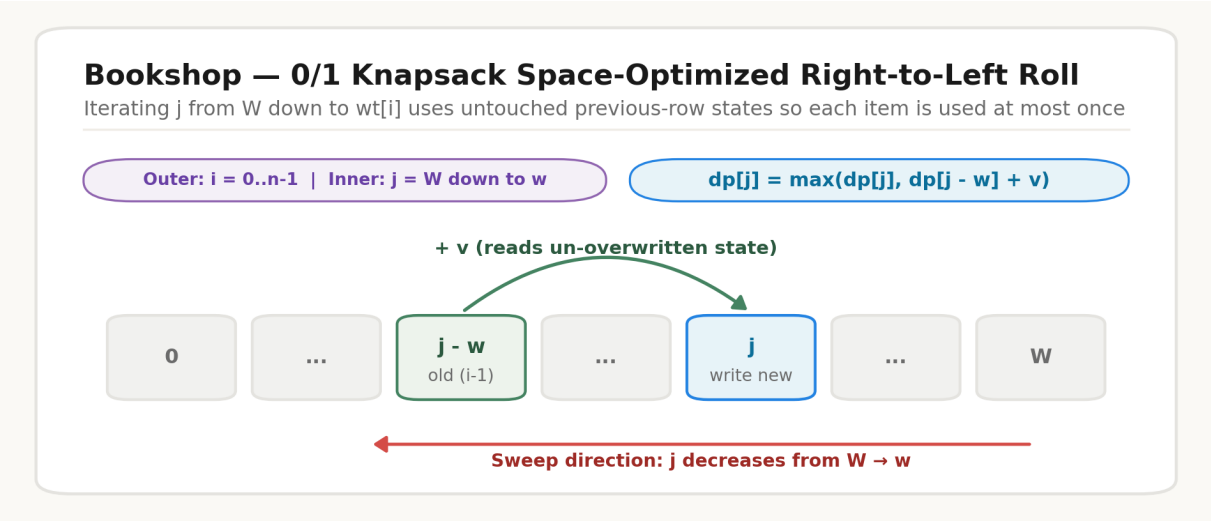


- **Definition**: $f(r, c)$ = number of valid paths from $(0, 0)$ to (r, c).
- **Base Case**: $f(0, 0) = 1$ if $(0, 0)$ is unblocked, else 0.
- **Transition**:

$$f(r, c) = \begin{cases} 0 & \text{if cell } (r, c) \text{ is blocked } (*) \\ (f(r-1, c) + f(r, c-1)) \bmod (10^9 + 7) & \text{otherwise} \end{cases}$$

7 Bookshop

Goal: 0/1 Knapsack (single-use items) — maximize total value subject to total weight $\leq W$. ($\mathcal{O}(nW) \approx 10^8$ operations).



- **Definition**: $f(i, w)$ = maximum value using first i items with total weight $\leq w$.
- **Base Case**: $f(0, w) = 0$ for all $w \geq 0$.
- **Transition**:

$$f(i, w) = \begin{cases} \max\left(f(i-1, w),\ \text{val}_i + f(i-1, w - \text{wt}_i)\right) & \text{if } w - \text{wt}_i \geq 0 \\ f(i-1, w) & \text{otherwise} \end{cases}$$

- **Implementation Trick**: Use a 1D `dp[W+1]` array and overwrite **from right to left** ($j = W$ down to wt_i) so each item is used at most once.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, W;
    cin >> n >> W;

    vector<int> wt(n), val(n);
    for (int &x : wt)
        cin >> x;
    for (int &x : val)
        cin >> x;

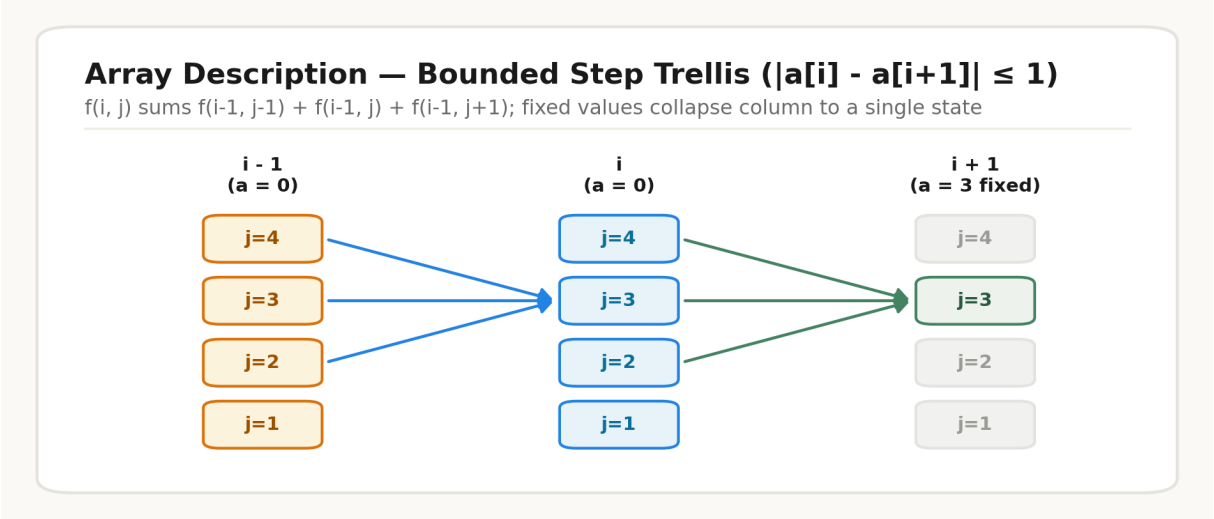
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        int w = wt[i], v = val[i];
        for (int j = W; j >= w; j--) {
            dp[j] = max(dp[j], dp[j - w] + v);
        }
    }

    cout << dp[W] << "\n";

    return 0;
}
```

8 Array Description

Goal: Given $a(i) \in \{0, \ldots, m\}$ ($m \leq 100$) where 0 means any value in $\{1, \ldots, m\}$ can be chosen, count valid arrays with $|a(i) - a(i+1)| \leq 1$.



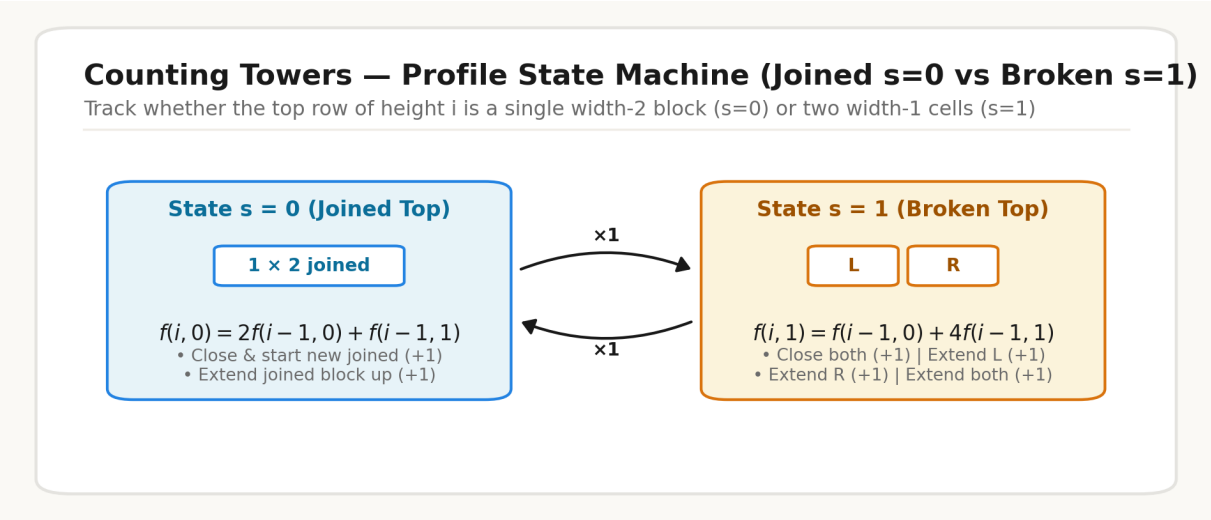
- **Definition**: $f(i, j)$ = ways to fill the first i values such that $a(i) = j$.
- **Base Case**: $f(1, j) = 1$ if $a(1) = 0$ or $a(1) = j$; otherwise 0.
- **Transition**:

$$f(i,j) = \begin{cases} \displaystyle\sum_{k=\max(1,j-1)}^{\min(m,j+1)} f(i-1,k) \pmod{10^9+7} & \text{if } a(i) = 0 \text{ or } a(i) = j \\ 0 & \text{otherwise} \end{cases}$$

- **Note**: Active window optimization improves average-case runtime; sparse window optimizations handle long runs of zeroes.

9 Counting Towers

Goal: Count ways to build a tower of height $h \times 2$ using unlimited integer-sized blocks.

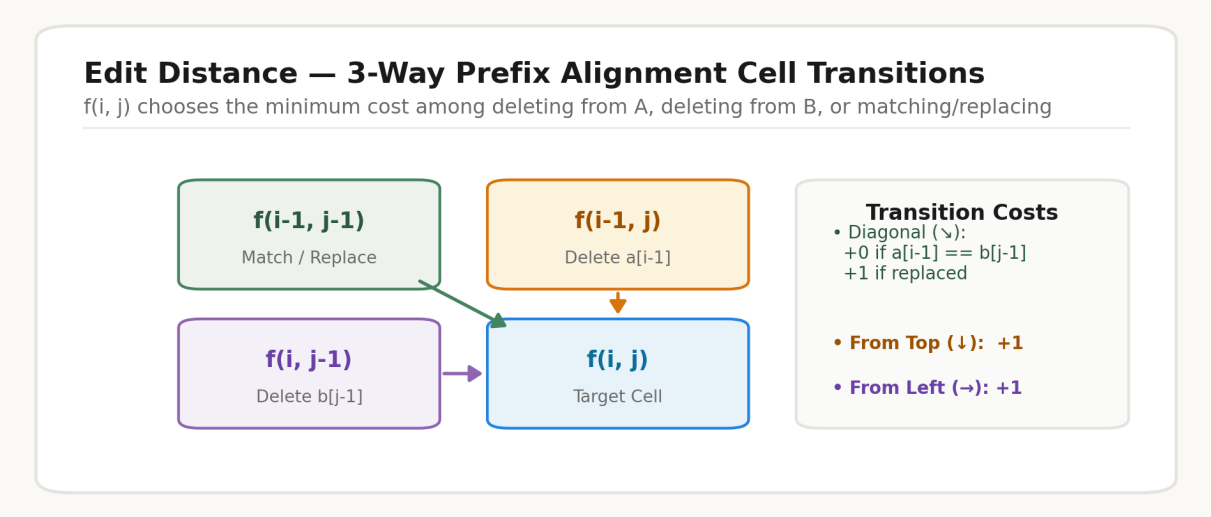


- **Definition**: $f(i, s)$ = ways to build up to height i (flat top) where row i is joined ($s = 0$) or broken ($s = 1$).
- **Base Case**: $f(1, 0) = 1$, $f(1, 1) = 1$.
- **Transition**:

$$f(i, 0) = \left(2\, f(i-1, 0) + f(i-1, 1)\right) \bmod (10^9 + 7)$$
$$f(i, 1) = \left(f(i-1, 0) + 4\, f(i-1, 1)\right) \bmod (10^9 + 7)$$

10 Edit Distance

Goal: Minimum operations (delete from a, delete from b, or convert a character) to align strings a and b.



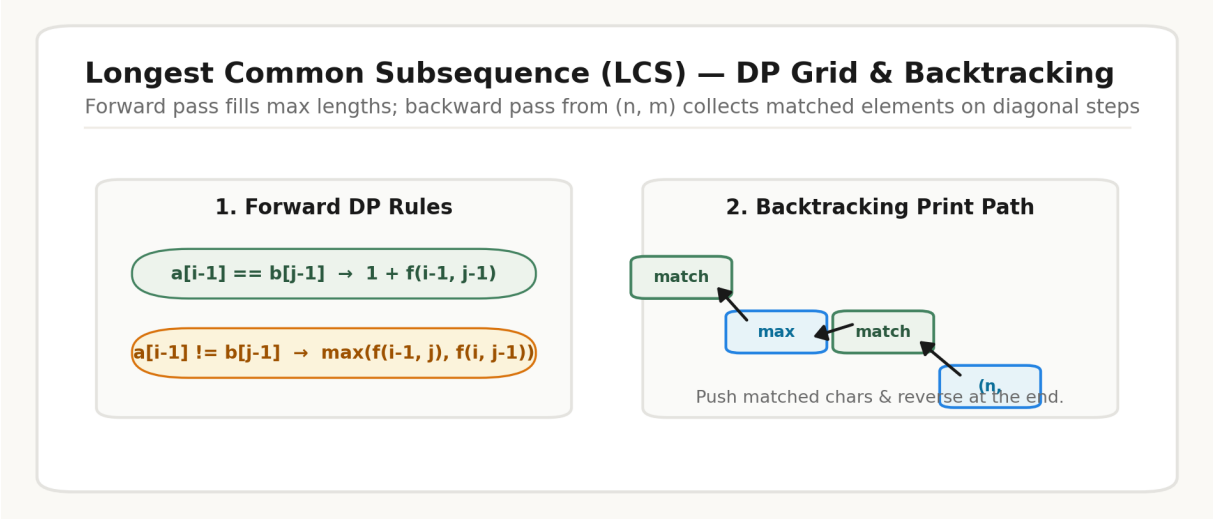
- **Definition**: $f(i, j)$ = edit distance between prefix of length i of a and prefix of length j of b.

- **Base Cases**: $f(0, j) = j$ and $f(i, 0) = i$.
- **Transition**:

$$f(i, j) = \min \Big\{ 1 + f(i-1, j),\ 1 + f(i, j-1),\ f(i-1, j-1) + \mathbf{1}[a(i-1) \neq b(j-1)] \Big\}$$

11 LCS (Longest Common Subsequence)

Goal: Compute the LCS length of arrays a and b and reconstruct the sequence.



- **Definition**: $f(i, j)$ = LCS length of prefixes $a[0..i-1]$ and $b[0..j-1]$.
- **Base Cases**: $f(i, 0) = f(0, j) = 0$.
- **Transition**:

$$f(i, j) = \begin{cases} 1 + f(i-1, j-1) & \text{if } a(i-1) = b(j-1) \\ \max\left(f(i-1, j),\ f(i, j-1)\right) & \text{otherwise} \end{cases}$$

- **Reconstruction**: Walk backward from (n, m): if $a[n-1] == b[m-1]$, push element and decrement both n, m; otherwise step toward the larger neighbor.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define in long long
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (int &x : a)
        cin >> x;
    for (int &x : b)
        cin >> x;
    vector<vector<in>> dp(n + 1, vector<in>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
```

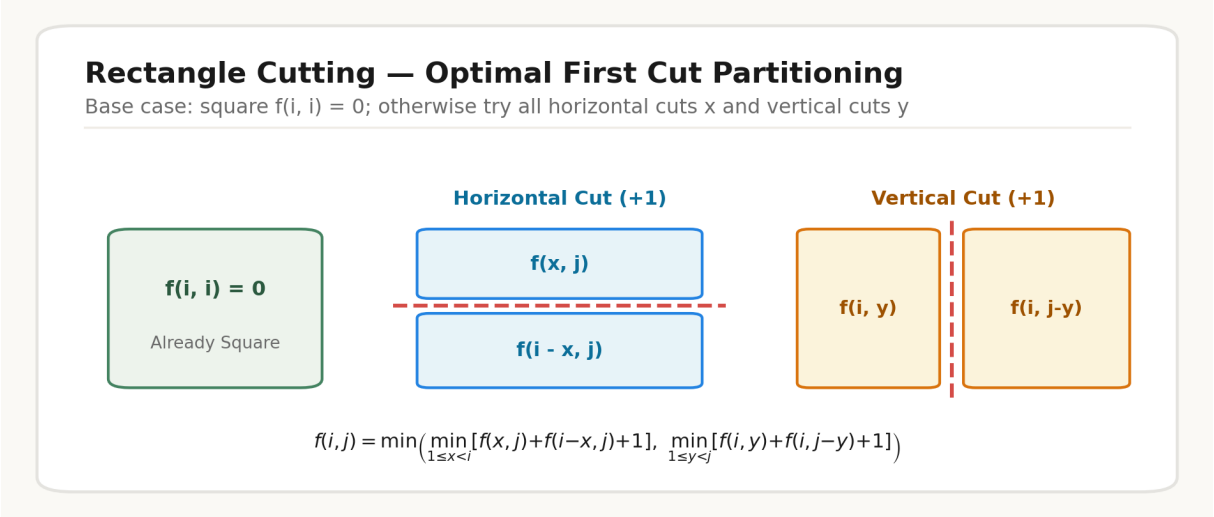
```
    }

    cout << dp[n][m] << "\n";

    vector<int> ans;
    while (dp[n][m] > 0) {
        if (a[n - 1] == b[m - 1]) {
            ans.push_back(a[n - 1]);
            n--;
            m--;
        } else if (dp[n - 1][m] > dp[n][m - 1]) {
            n--;
        } else {
            m--;
        }
    }
    reverse(ans.begin(), ans.end());
    for (int x : ans)
        cout << x << " ";
    cout << "\n";
#undef int
}
```

12 Rectangle Cutting

Goal: Minimum horizontal/vertical cuts to partition an $a \times b$ rectangle into squares. Complexity: $\mathcal{O}(ab(a+b))$.



- **Definition**: $f(i,j)$ = minimum cuts for an $i \times j$ rectangle.
- **Base Cases**: $f(i,i) = 0$; initialize non-square states to $+\infty$.
- **Transition** (for $i \neq j$):

$$f(i,j) = \min\left\{ \min_{1\le x<i} \left(f(x,j) + f(i-x,j) + 1\right),\ \min_{1\le y<j} \left(f(i,y) + f(i,j-y) + 1\right) \right\}$$

```
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n, m;
```

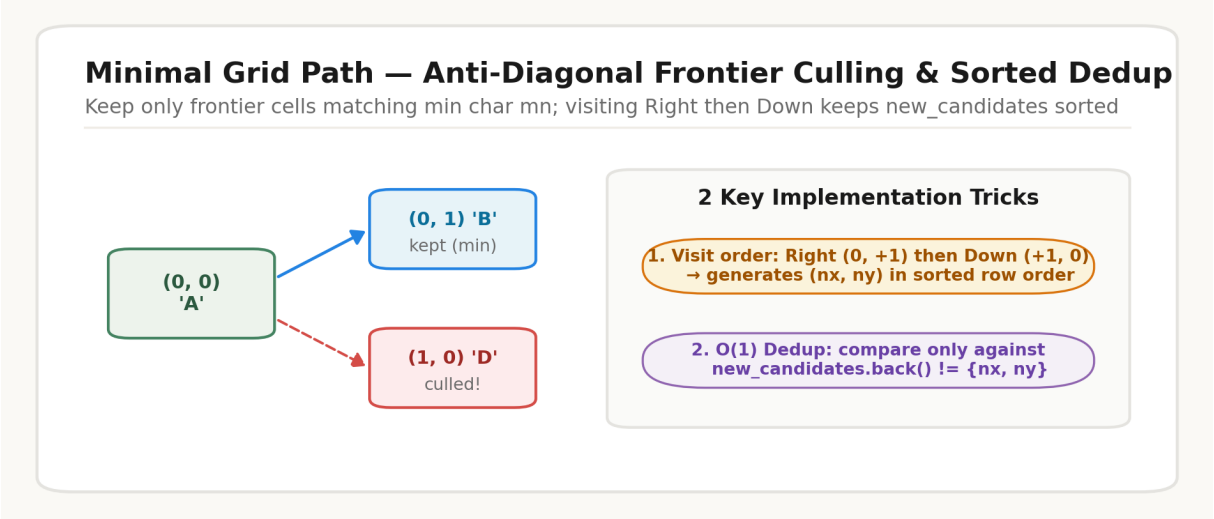
```
    cin >> n >> m;
    const int inf = (int)1e18;
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (i == j) {
                dp[i][j] = 0;
            } else {
                dp[i][j] = inf;
                for (int k = 1; k < i; k++) {
                    dp[i][j] = min(dp[i][j], dp[k][j] + dp[i - k][j] + 1);
                }
                for (int k = 1; k < j; k++) {
                    dp[i][j] = min(dp[i][j], dp[i][k] + dp[i][j - k] + 1);
                }
            }
        }
    }
    cout << dp[n][m] << "\n";

#undef int
}
```

13 Minimal Grid Path

Goal: Find the lexicographically smallest string path from top-left to bottom-right in an $n \times n$ character grid.



- **Greedy BFS / Anti-Diagonal Frontier Culling**: Maintain active frontier `candidates`. At each of the $2n-1$ steps, find $\text{mn} = \min_{(x,y)\in\text{candidates}} v[x][y]$, output mn, and expand only cells with $v[x][y] == \text{mn}$.
- **Two Key Tricks**:
  1. Visit **Right** $(0,+1)$ then **Down** $(+1,0)$ so `new_candidates` is naturally generated in sorted order.
  2. Because coordinates are generated in sorted order, deduplicate in $\mathcal{O}(1)$ by checking `new_candidates.back() != make_pair(nx, ny)`.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n;
    cin >> n;
    vector<string> v(n);
    for (auto &x : v)
        cin >> x;

    vector<pair<int, int>> candidates, new_candidates;
    candidates.push_back({0, 0});

    // order matters, right then down makes (0,0)->(0,1)->(1,1)->(1,2) which is
    // sorted
    vector<int> dx = {0, 1};
    vector<int> dy = {1, 0};

    int steps = 2 * n - 1;
    while (steps--) {
        char mn = 'Z' + 1;
        int sz = (int)candidates.size();
        for (auto &[x, y] : candidates) {
            mn = min(mn, v[x][y]);
        }

        cout << mn;

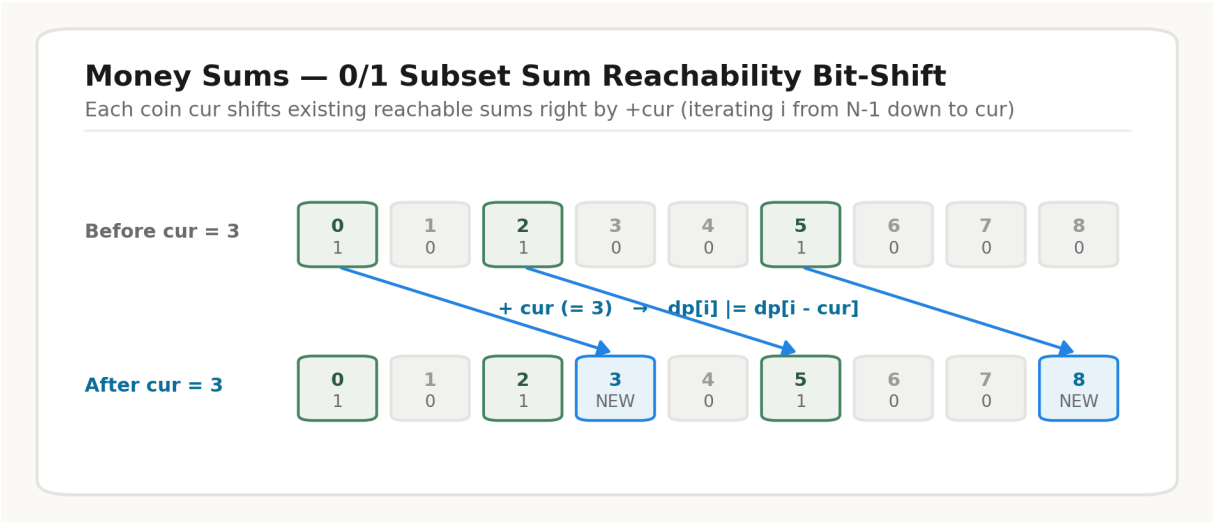
        new_candidates.clear();
        for (auto &[x, y] : candidates) {
            if (v[x][y] != mn)
                continue;
            for (int dir = 0; dir < 2; dir++) {
                int nx = x + dx[dir];
                int ny = y + dy[dir];
                if (nx < n && ny < n &&
                    (new_candidates.empty() ||
                     new_candidates.back() != make_pair(nx, ny))) {
                    new_candidates.push_back({nx, ny});
                }
                // at each step there can be too many duplicates so filter those
            }
        }

        swap(candidates, new_candidates);
    }
    cout << '\n';

#undef int
}
```

14 Money Sums

Goal: Find all distinct sums achievable from subsets of n integers ($n \leq 100, a_i \leq 1000$).



- **Definition**: $f(s) \in \{0, 1\}$ indicates whether sum s is possible.
- **Base Case**: $f(0) = 1$.
- **Transition** (*for each coin cur, for target* $i = N - 1$ *down to cur*):

$$f(i) \mid= f(i - \text{cur})$$

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n;
    cin >> n;
    vector<int> v(n);
    for (int &x : v)
        cin >> x;

    const int N = (int)1e5 + 5;
    vector<int> dp(N, 0);
    dp[0] = 1;

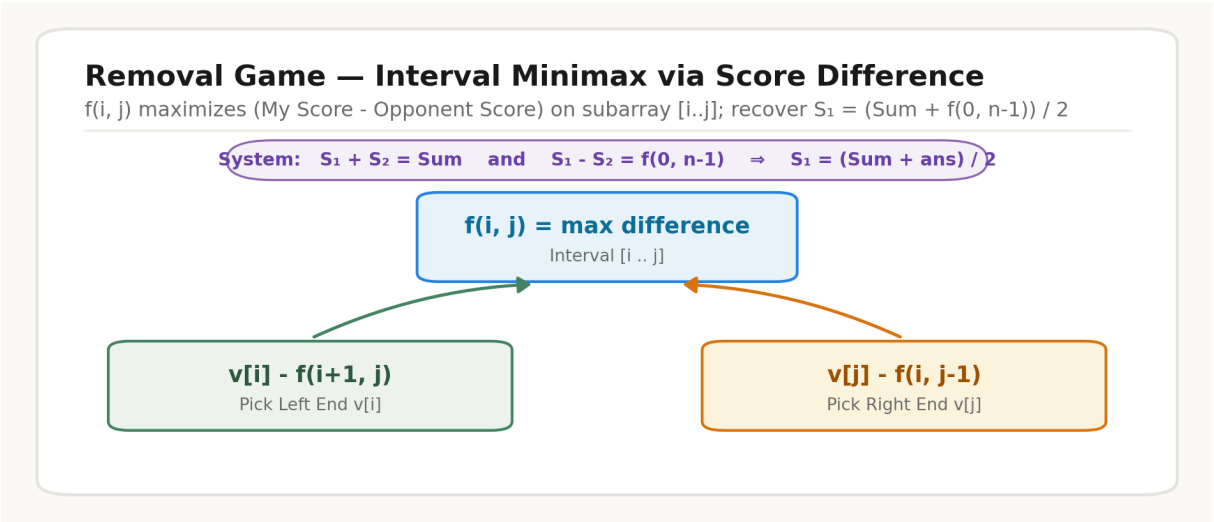
    int ans = 0;
    for (int cur : v) {
        for (int i = N - 1; i >= cur; i--) {
            if (dp[i - cur] && !dp[i])
                dp[i] = 1, ans++;
        }
    }

    cout << ans << "\n";
    for (int i = 1; i < N; i++) {
        if (dp[i]) {
            cout << i << " ";
        }
    }
    cout << "\n";

#undef int
}
```

15 Removal Game

Goal: Two players remove elements from either end of array v; find the first player's optimal score.



- **Zero-Sum Minimax Difference Trick**: Maximize $\Delta = (\text{my score} - \text{other's score})$ on subarray $[i..j]$.
- **Definition**: $f(i,j) = \max(S_{\text{cur}} - S_{\text{other}})$ on segment $i \dots j$.
- **Base Case**: $f(i,i) = v(i)$.
- **Transition** (*fill from small to large segment lengths* $len = 2 \dots n$):

$$f(i,j) = \max\{\, v(i) - f(i+1,j),\ v(j) - f(i,j-1) \,\}$$

- **Score Recovery**: Since $S_1 + S_2 = \text{sum}$ and $S_1 - S_2 = f(0,n-1)$, we get $S_1 = \dfrac{\text{sum} + f(0,n-1)}{2}$.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n;
    cin >> n;
    vector<int> v(n);
    for (int &x : v)
        cin >> x;

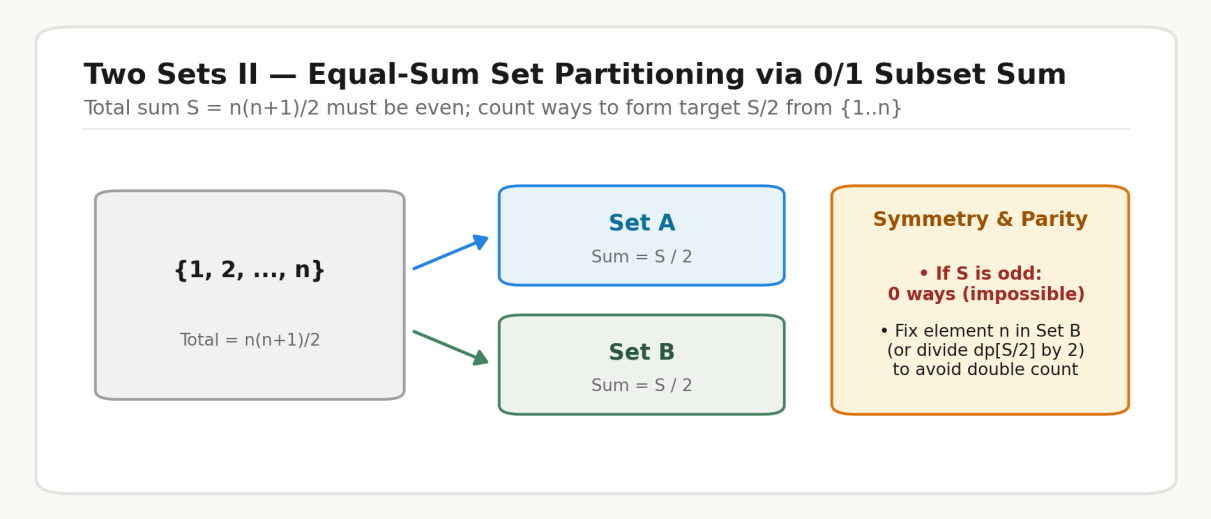
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++)
        dp[i][i] = v[i];
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            dp[i][j] = max(v[i] - dp[i + 1][j], v[j] - dp[i][j - 1]);
        }
    }

    int ans = dp[0][n - 1];
    int sum = accumulate(v.begin(), v.end(), 0LL);
    cout << (sum + ans) / 2 << "\n";

#undef int
}
```

16 Two Sets 2

Goal: Count ways to partition $\{1, \ldots, n\}$ into two sets of equal sum.

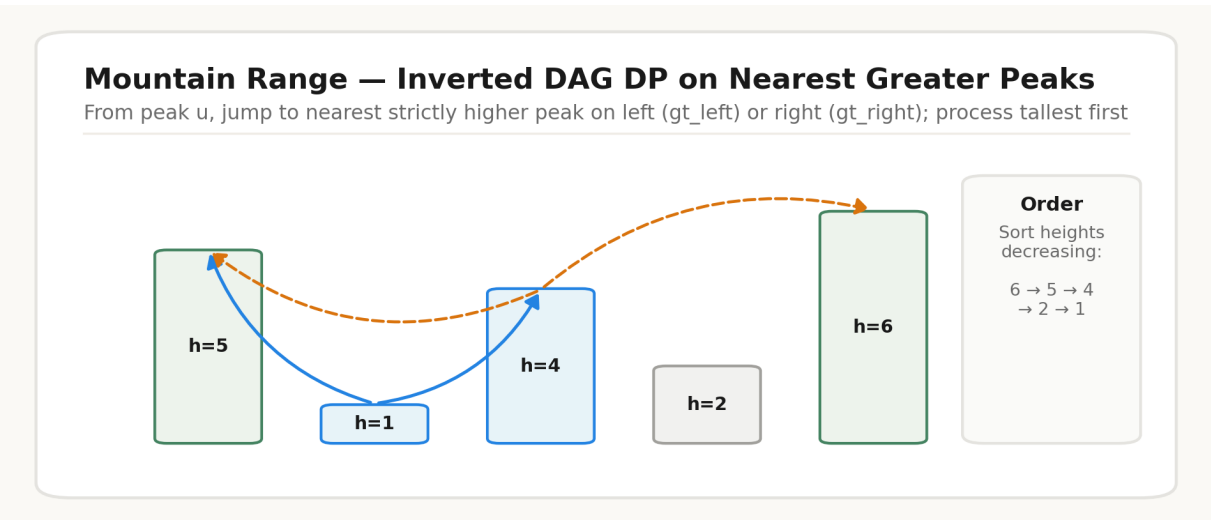


- **No-Solution Check**: Total sum $S = \frac{n(n+1)}{2}$. If S is odd, output 0.
- **0/1 Subset Sum Formulation**: Target sum $T = S/2$, base $f(0) = 1$, and for each $x \in \{1, \ldots, n-1\}$ (fixing n in Set 2 to avoid double counting) and $s = T$ down to x:

$$f(s) = \big(f(s) + f(s-x)\big) \bmod (10^9 + 7)$$

17 Mountain Range

Goal: DP on Cartesian Tree / Monotonic Stack DAG by processing heights in decreasing order.



- **Inversion of State**: Instead of branching from a tall peak to all smaller peaks, jump from peak u upward to the nearest strictly higher peak on the left (gt_left[u]) or right (gt_right[u]).
- **Definition**: $\mathrm{dp}(u) =$ max range ending at u, with base $\mathrm{dp}(u) = 1$.
- **Transition** (*sorted in decreasing order of height*):

$$\mathrm{dp}(u) = \max\{\, \mathrm{dp}(u),\ 1 + \mathrm{dp}(\mathrm{gt_left}[u]),\ 1 + \mathrm{dp}(\mathrm{gt_right}[u]) \,\}$$

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n;
    cin >> n;
    vector<pair<int, int>> v(n);
    for (int i = 0; i < n; i++) {
        cin >> v[i].first;
        v[i].second = i;
    }

    auto mono_stack = [](const auto &arr, auto first, auto last, auto should_pop) {
        int n = arr.size();
        vector<int> res(n, -1);
        vector<int> st;

        for (auto it = first; it != last; ++it) {
            while (!st.empty() && should_pop(arr[st.back()].first, (*it).first)) {
                st.pop_back();
            }
            int i = &(*it) - arr.data();

            if (!st.empty())
                res[i] = st.back();
            st.push_back(i);
        }
        return res;
    };

    auto gt_left = mono_stack(v, v.begin(), v.end(), less_equal<int>());
    auto gt_right = mono_stack(v, v.rbegin(), v.rend(), less_equal<int>());

    sort(v.begin(), v.end(), greater<>());

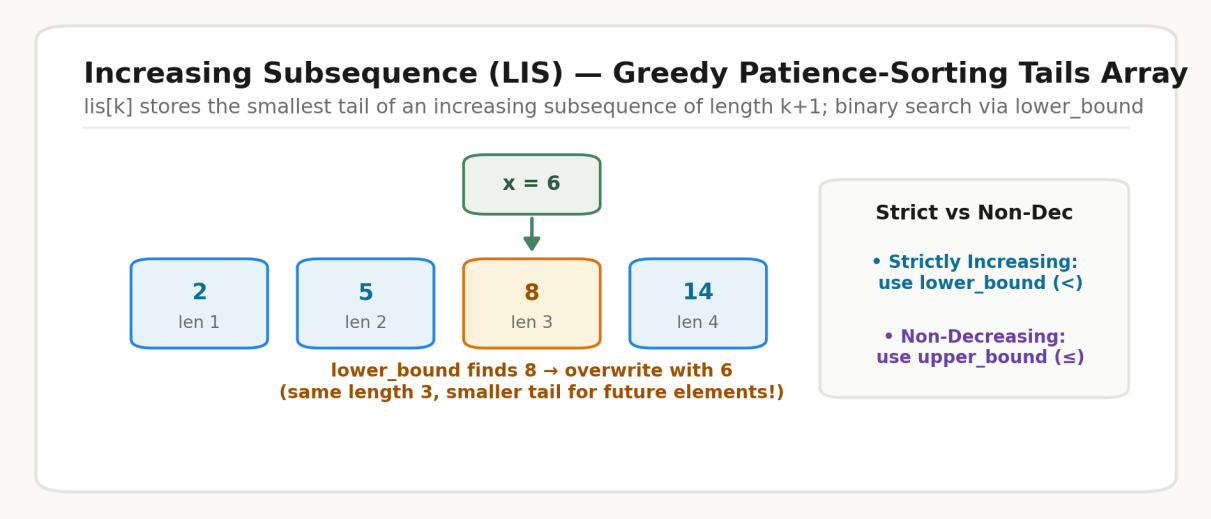
    vector<int> ans(n, 1);
    for (auto [h, idx] : v) {
        if (gt_left[idx] != -1) {
            ans[idx] = max(ans[idx], 1 + ans[gt_left[idx]]);
        }
        if (gt_right[idx] != -1) {
            ans[idx] = max(ans[idx], 1 + ans[gt_right[idx]]);
        }
    }

    cout << *max_element(ans.begin(), ans.end()) << endl;

#undef int
}
```

18 Increasing Subsequence

Goal: Longest Increasing Subsequence (LIS) in $\mathcal{O}(n \log n)$ via greedy tails array.



- **Definition**: $f(u) =$ minimum ending element of an increasing subsequence of length $u + 1$.
- **Transition**: For each x, binary search via `lower_bound`; append if at end, else overwrite $*it = x$ (for longest non-decreasing subsequence, use `upper_bound`).

```cpp
#include <algorithm>
#include <bits/stdc++.h>
using namespace std;

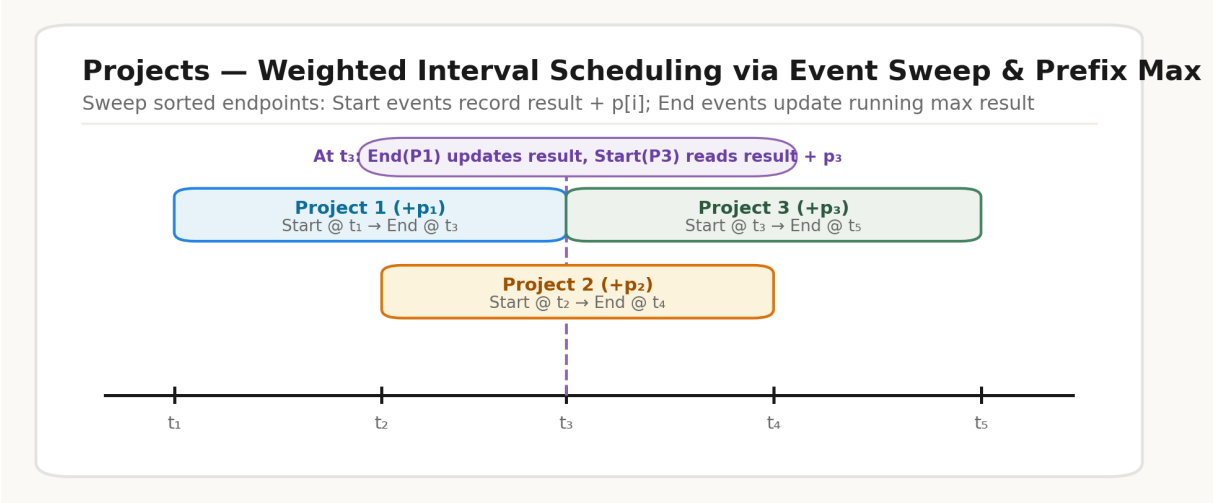
int main() {
#define int long long
    int n;
    cin >> n;
    vector<int> v(n);
    for (int &x : v)
        cin >> x;

    vector<int> lis;
    for (int x : v) {
        auto it = lower_bound(lis.begin(), lis.end(), x);
        if (it == lis.end()) {
            lis.push_back(x);
        } else {
            *it = x;
        }
    }

    cout << (int)lis.size() << "\n";
#undef int
}
```

19 Projects

Goal: Weighted interval scheduling via prefix maximum optimization and coordinate-compressed event sweep.



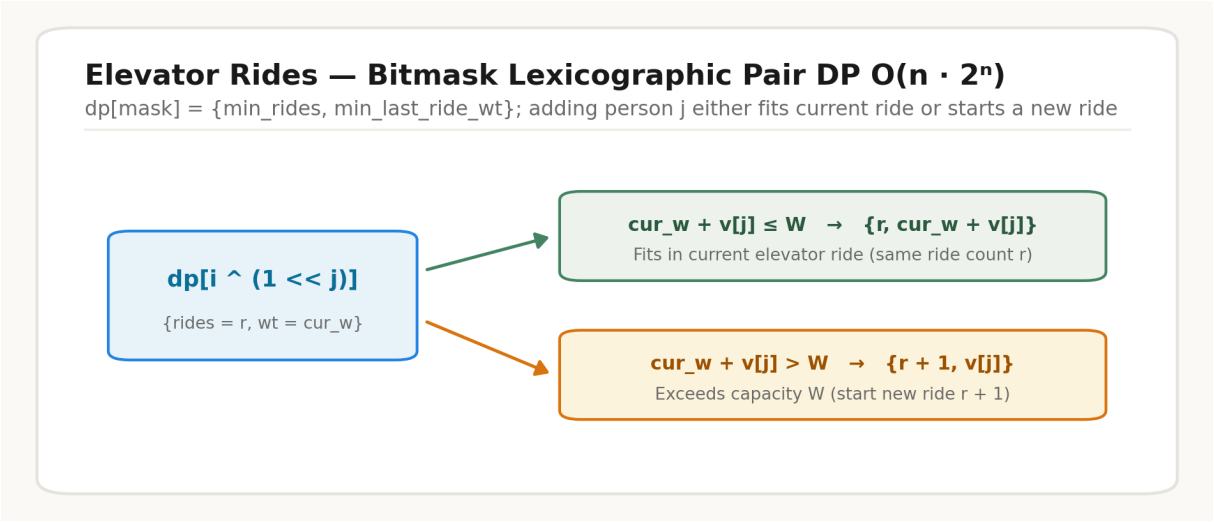
- **Pattern**: $f(j) = \max_{i \le \text{limit}} f(i) + p_j$. Process sorted start/end events:

$$\begin{aligned} \text{Start event } (t, p, 0): &\quad \text{money}[p] = \text{result} + p[p] \\ \text{End event } (t, p, 1): &\quad \text{result} = \max(\text{result},\ \text{money}[p]) \end{aligned}$$

```
for (auto [time, project, is_end] : events) {
    if (is_end) {
        result = max(result, money[project]);
    } else {
        money[project] = result + p[project];
    }
}
```

20 Elevator Rides

Goal: Minimum elevator rides of capacity W for $n \le 20$ people ($\mathcal{O}(3^n) \to \mathcal{O}(n2^n)$ bitmask DP).




```
for (int m = 0; m < (1 << N); ++m) {
    for (int s = m; s > 0; s = (s - 1) & m) {
        // empty subset is not here
    }
}
```

- **Definition**: $f(\text{mask}) = \{\text{min rides, min last ride weight}\}$, base $f(0) = \{1, 0\}$.
- **Transition**: For each person $j \in \text{mask}$ with $\text{prev} = f(\text{mask} \oplus 2^j)$:

$$f(\text{mask}) = \min_{j \in \text{mask}} \begin{cases} \{\text{prev}[0],\ \text{prev}[1] + v_j\} & \text{if } \text{prev}[1] + v_j \le W \\ \{\text{prev}[0] + 1,\ v_j\} & \text{otherwise} \end{cases}$$

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n, w;
    cin >> n >> w;
    vector<int> v(n);
    for (int &x : v)
        cin >> x;

    const int INF = (int)1e18 + 5;
    vector<array<int, 2>> dp(1ll << n, {INF, INF});

    // f(i) = {min rides, min last ride wt}
    dp[0] = {1, 0}; // extend last keeping rides as 1

    for (int i = 1; i < (1ll << n); i++) {
        for (int j = 0; j < n; j++) {
            // skip if unset
            if (!((i >> j) & 1))
                continue;

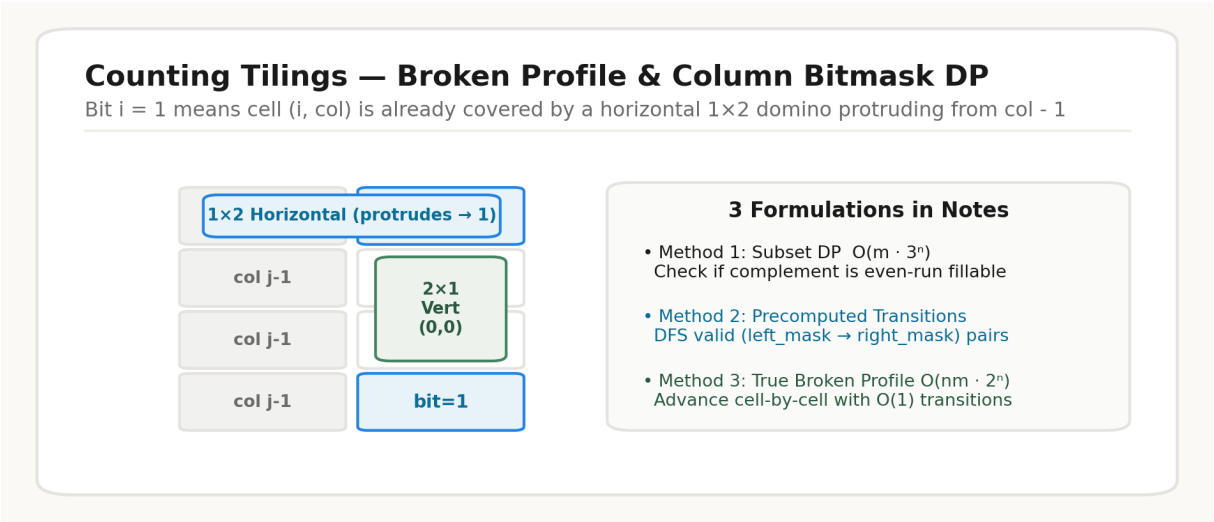
            auto prev = dp[i ^ (1ll << j)];
            if (prev[1] + v[j] <= w) {
                // no new ride
                dp[i] = min(dp[i], {prev[0], prev[1] + v[j]});
            } else {
                dp[i] = min(dp[i], {prev[0] + 1, v[j]});
            }
        }
    }

    cout << dp[(1ll << n) - 1][0] << "\n";

#undef int
}
```

21 Counting Tilings

Goal: Broken profile / column bitmask DP to tile an $n \times m$ grid ($n \le 10, m \le 1000$) with 1×2 and 2×1 dominoes.



- **Definition**: $f(j, \text{mask})$ = ways to fill through column $j-1$ where bit $i = 1$ means cell (i, j) is covered from column $j-1$.
- **Method 1 — Subset DP $\mathcal{O}(m3^n)$**:

$$f(j, \text{profile}) = \sum_{\substack{\text{sub} \subseteq \overline{\text{profile}} \\ \text{fillable}(\overline{\text{profile}} \oplus \text{sub})}} f(j-1, \text{sub}) \pmod{10^9+7}$$

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n, m;
    cin >> n >> m;

    vector<int> cur(1ll << n, 0), next(1ll << n, 0);
    cur[0] = 1; // clean profile is 1 way

    // prcompute what masks are fillable by vertical cols
    // basically what bits are vertically fillable
    vector<bool> fillable(1ll << n, false);
    for (int mask = 0; mask < (1ll << n); mask++) {
        bool can = true;
        // idea is that seq of one bits should be even length
        int len = 0;
        for (int i = 0; i < n; i++) {
            if (mask & (1ll << i)) {
                len++;
            } else {
                if (len & 1) {
                    can = false;
                    break;
                }
                len = 0;
            }
        }
        // last seq
        if (len & 1)
            can = false;
        fillable[mask] = can;
    }

    const int mod = (int)1e9 + 7;
```

```
    for (int j = 0; j < m; j++) { // for each col
        fill(next.begin(), next.end(), 0);
        for (int profile = 0; profile < (1ll << n); profile++) { // for each profile
            int prev_profile = ((1ll << n) - 1) ^ profile;        // inverted

            for (int sub = prev_profile;; sub = (sub - 1) & prev_profile) {
                // submasks of prev_profile, what's not filled should be
                // fillable by vertical cols

                int vertical = prev_profile ^ sub;
                if (fillable[vertical]) {
                    (next[profile] += cur[sub]) %= mod;
                }

                // submask iteration that processes 0 submask too
                if (sub == 0)
                    break;
            }
        }
        cur.swap(next);
    }

    cout << cur[0] << "\n";

#undef int
}
```

Method 2 — Precomputed Transitions:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n, m;
    cin >> n >> m;

    vector<int> cur(1ll << n, 0), next(1ll << n, 0);
    cur[0] = 1; // clean profile is 1 way
    const int mod = 1e9 + 7;

    // precomp transitions that are valid ( left mask -> right mask )
    vector<pair<int, int>> trans;
    auto gen = [&](auto &&f, int row, int left, int right) -> void {
        if (row > n)
            return;
        if (row == n) {
            trans.emplace_back(left, right);
            return;
        };
        // 1 => is this filled by extension from prev col
        // 1x2 place, left 0, right filled by extension
        f(f, row + 1, left, right | (1ll << row));
        // another valid is when left acceps an extension, in this case right is 0
        // as we place a new 1x2 here
        f(f, row + 1, left | (1ll << row), right);
        // vertical 2x1, set both as 0
        f(f, row + 2, left, right);
    };
    gen(gen, 0, 0, 0);

    for (int j = 0; j < m; j++) { // for each col
        fill(next.begin(), next.end(), 0);
        for (auto &[left, right] : trans) {
            (next[right] += cur[left]) %= mod;
        }
        swap(cur, next);
```

```
    }

    cout << cur[0] << "\n";

#undef int
}
```

Method 3 — Actual Broken Profile $\mathcal{O}(nm2^n)$:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
#define int long long
    int n, m;
    cin >> n >> m;

    vector<int> cur(1ll << n, 0), next(1ll << n, 0);
    cur[0] = 1;

    const int mod = 1e9 + 7;

    for (int j = 0; j < m; j++) {
        for (int i = 0; i < n; i++) {
            fill(next.begin(), next.end(), 0);
            for (int mask = 0; mask < (1ll << n); mask++) {
                if (!cur[mask]) // not reachable
                    continue;

                // blocked => next is filled via col-1 (cur)
                // so 1 here = do I fill next col too?
                bool blocked = (mask & (1ll << i)) != 0;
                if (blocked) {
                    int next_mask = mask ^ (1ll << i); // must be 0
                    (next[next_mask] += cur[mask]) %= mod;
                } else {
                    // two choices, place vertical or horizontal

                    // horizontal
                    int next_mask = mask | (1ll << i); // next mask if filled by col-1, aka cur
                    (next[next_mask] += cur[mask]) %= mod;

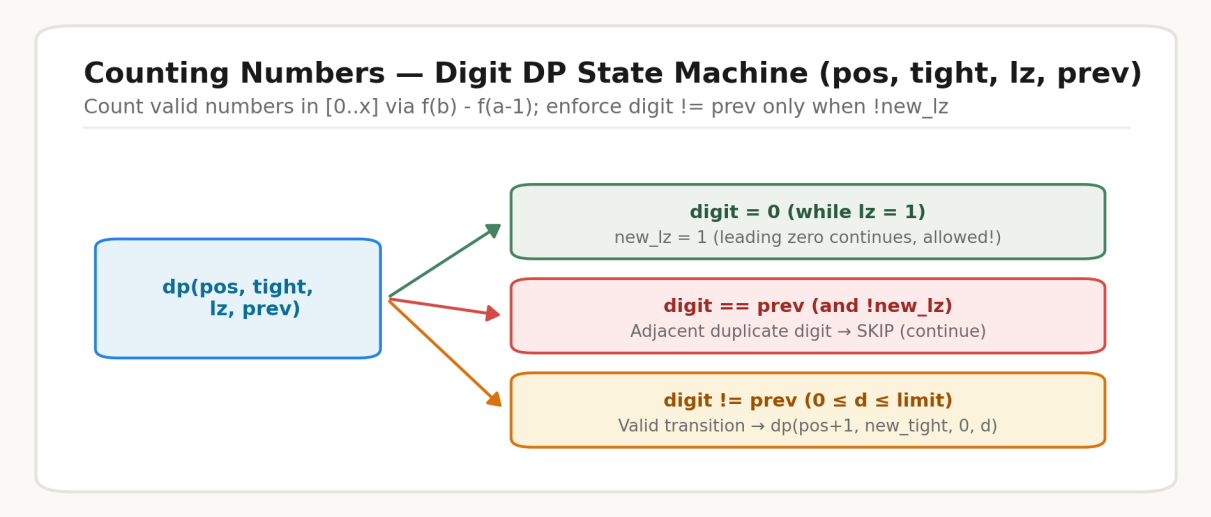
                    // cur is a valid mask for vertical placement
                    if (i + 1 < n && (mask & (1ll << (i + 1))) == 0) {
                        next_mask = mask; // it isn't filled at all
                        // i+1 needs to be marked as filled so that loop
                        // skips it
                        next_mask |= (1ll << (i + 1));
                        (next[next_mask] += cur[mask]) %= mod;
                    }
                }
            }
            cur.swap(next);
        }
    }

    cout << cur[0] << "\n";

#undef int
}
```

22 Counting Numbers

Goal: Count numbers in $[a, b]$ with no two adjacent equal digits via Digit DP $f(b) - f(a-1)$.



- **Transition**: With $\text{limit} = \text{tight}?(s[\text{pos}] - \text{'0'}) : 9$,

$$\text{dp}(\text{pos}, \text{tight}, \text{lz}, \text{prev}) = \sum_{\substack{d=0 \\ \neg(\neg\text{new_lz} \wedge d = \text{prev})}}^{\text{limit}} \text{dp}(\text{pos}+1,\ \text{tight} \wedge (d = \text{limit}),\ \text{lz} \wedge (d = 0),\ d)$$

General Digit DP Template:

```
int dp(int pos, bool tight, bool lz, int state) {
    if (pos == S.size()) return 1;
    if (MEMO[pos][tight][lz][state] != -1) return MEMO[pos][tight][lz][state];

    int limit = tight ? (S[pos] - '0') : 9;
    int ans = 0;

    for (int digit = 0; digit <= limit; digit++) {
        bool new_tight = tight && (digit == limit);
        bool new_lz = lz && (digit == 0);

        int new_state = state;
        if (!new_lz) {
            // new_state update logic
        }

        ans += dp(pos + 1, new_tight, new_lz, new_state);
    }

    return MEMO[pos][tight][lz][state] = ans;
}
```

Solution Code:

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

int MEMO[20][2][2][10];
string s;

int dp(int pos, bool tight, bool lz, int prev) {
    if (pos == (int)s.size())
        return 1;
    if (MEMO[pos][tight][lz][prev] != -1)
        return MEMO[pos][tight][lz][prev];
```

```
    int limit = tight ? (s[pos] - '0') : 9;
    int ans = 0;

    for (int digit = 0; digit <= limit; digit++) {
        bool new_tight = tight && (digit == limit);
        bool new_lz = lz && (digit == 0);

        // !new_lz => I placed some digit that makes it non leading zero
        if (digit == prev && !new_lz)
            continue;
        ans += dp(pos + 1, new_tight, new_lz, digit);
    }

    return MEMO[pos][tight][lz][prev] = ans;
}

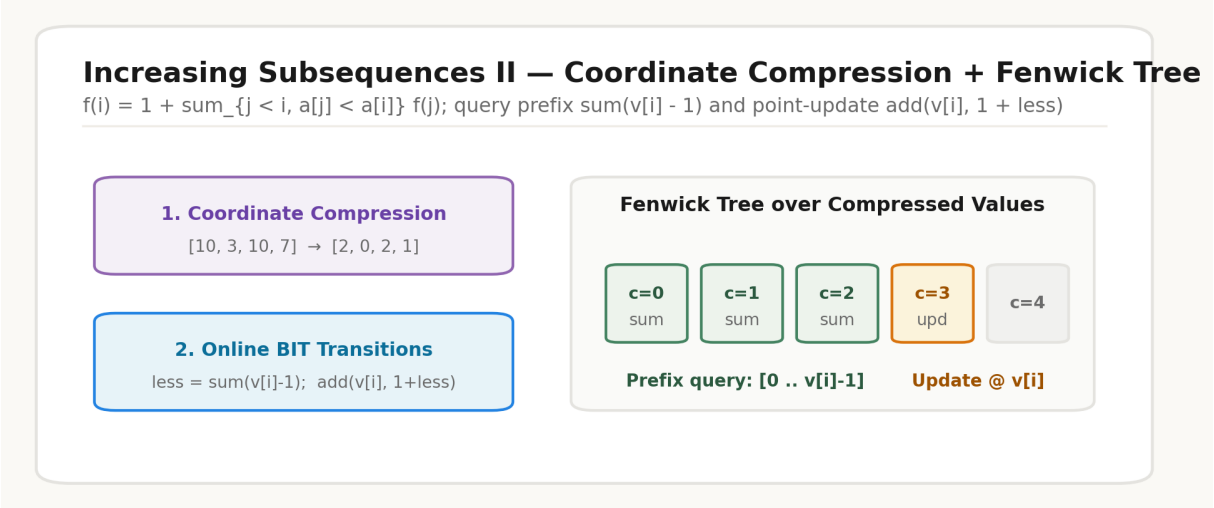
signed main() {
    int a, b;
    cin >> a >> b;

    auto f = [&](int x) {
        if (x < 0)
            return 0ll;
        if (x == 0)
            return 1ll;
        s = to_string(x);
        memset(MEMO, -1, sizeof(MEMO));
        return dp(0, 1, 1, 0);
    };

    cout << f(b) - f(a - 1) << "\n";
}
```

23 Increasing Subsequences 2

Goal: Count strictly increasing subsequences modulo $10^9 + 7$ using Coordinate Compression + Fenwick Tree (BIT).



- **Definition**: $f(i)$ = count of increasing subsequences ending at index i.
- **Base Case**: $f(i) = 1$ (single element).

- **Transition**:

$$f(i) = \left(1 + \sum_{j<i,\ a(j)<a(i)} f(j)\right) \bmod (10^9+7) = (1 + \text{BIT.sum}(v_i - 1)) \bmod (10^9+7)$$

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long

signed main() {
    int n;
    cin >> n;
    vector<int> v(n);
    for (int &x : v)
        cin >> x;

    // cnt doesn't change on compression
    auto compress = [&](vector<int> &arr) {
        vector<int> sorted = arr;
        sort(sorted.begin(), sorted.end());
        sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
        for (int &x : arr) {
            x = lower_bound(sorted.begin(), sorted.end(), x) - sorted.begin();
        }
    };
    compress(v);

    const int mod = 1e9 + 7;
    vector<int> ft(n + 1, 0);
    auto add = [&](int i, int val) {
        for (++i; i <= n; i += i & -i)
            (ft[i] += val) %= mod;
    };
    auto sum = [&](int i) {
        int res = 0;
        for (++i; i > 0; i -= i & -i)
            (res += ft[i]) %= mod;
        return res;
    };

    int ans = 0;
    for (int i = 0; i < n; i++) {
        int less = sum(v[i] - 1);
        (ans += 1 + less) %= mod;
        add(v[i], 1 + less);
    }

    cout << ans << '\n';
}
```